Factorizer: A Scalable Interpretable Approach to Context Modeling for Medical Image Segmentation

Pooya Ashtari *[a] and Sabine Van Huffel[a]

[a] *Department of Electrical Engineering (ESAT), KU Leuven*

Abstract

Medical image segmentation is crucial for various clinical purposes. The vast majority of state-of-the-art segmentation models are Convolutional Neural Networks (CNNs) comprising encoder and decoder paths with skip connections in between. However, the inherent locality of convolution makes these models fail to effectively exploit long-range spatial dependencies, essential for better recognition of some structures, e.g., brain lesions. Transformers have recently proved promising performance on vision tasks, including semantic segmentation, mostly due to their capability of modeling global contexts. Nevertheless, lack of locality inductive bias makes Transformers underperform their CNN counterparts in low-data regimes, which is usually the case in medical imaging. Moreover, the quadratic complexity of attention makes exiting Transformer-based models use self-attention layers only after somehow reducing the image resolution, thus fail to fully exploit global contexts present at higher resolutions. Hence, this work introduces a family of models, dubbed Factorizer, which leverage the power of low-rank matrix factorization for constructing an end-to-end segmentation model. Specifically, we propose a linearly scalable approach to context modeling by formulating Nonnegative Matrix Factorization (NMF) as a differentiable layer integrated into a U-shaped architecture. The shifted window technique is also utilized in combination with NMF to effectively aggregate local information. Built upon matrix factorization, Factorizers compete favorably with CNNs and Transformers in terms of accuracy, scalability, and interpretability, achieving state-of-the-art results on the BraTS dataset for brain tumor segmentation. Highly meaningful NMF components give a great interpretability advantage to Factorizers over CNNs and Transformers. Moreover, our ablation studies reveal an interesting distinctive feature of Factorizers that enables a significant speed-up in inference for a trained Factorizer with no extra steps and without sacrificing much accuracy.

Code: https://github.com/pashtari/factorizer

Keywords: Medical image segmentation, low-rank matrix factorization, convolution-free model.

1. Introduction

Medical image segmentation is an essential prerequisite for the analysis of anatomical structures and various clinical purposes, including diagnosis and treatment planning.

*Corresponding author. Email: pooya.ashtari@esat.kuleuven.be

The vast majority of effective segmentation models are based on Convolutional Neural Networks (CNNs). In particular, U-Net [1], consists of encoder and decoder parts with skip connections in between, has state-of-the-art performance on various medical image segmentation benchmarks. In a typical U-Net [1, 2], the encoder learns low-resolution contextual representation by progressively downsampling feature maps, while the decoder progressively upsamples the low-resolution feature maps to propagate contextual information to the higher-resolution layers. Moreover, skip connections from encoder layers to those of decoder at equal resolutions helps recover spatial information lost during downsampling.

Currently, proposed U-Net models mostly rely on convolution operations with small receptive fields, which are capable of exploiting only local context at each resolution. Hence, they generally fail to effectively model long-range spatial dependencies, often necessary for better recognition of some region semantics, e.g., tumors with variable shapes and sizes. Several works [3, 4] have employed dilated convolution for expanding the receptive fields. Nevertheless, the learning capabilities of convolutional layers are still limited due to their inherent locality. As a solution, integrating self-attention modules into CNNs [5, 6] has been proposed to enhance the capability of modeling non-local context.

Transformers [10] have achieved state-of-the-art performance on various Natural Language Processing (NLP) tasks. The attention mechanism enables Transformers to effectively model the pairwise interactions between the words in a sentence. Recently, Transformer-based models have been applied to vision tasks and proved promising. Specifically, Vision Transformer (ViT) [11] outperformed state-of-the-art CNNs on image recognition by large-scale pre-training and fine-tuning a pure Transformer. Unlike CNNs, ViT encodes images as a sequence of 1D patch embeddings (known as tokens) and dynamically highlights the important tokens using self-attention layers, which in turn increases the capability of learning long-range dependencies. Due to lack of locality inductive bias, ViT is data-hungry and generally requires a larger dataset to perform as effectively as its CNN counterparts, leading to poor performance when trained on insufficient data, which is usually the case in medical imaging. Furthermore, the quadratic complexity of self-attention makes Transformers computationally intractable on long sequences of patches. Therefore, exiting models use self-attention layers only after somehow reducing the image resolution, thereby failing to fully exploit the global context at the higher resolutions.

This work proposes a series of architectures, dubbed as Factorizer, which leverage the power of low-rank matrix factorization to construct an end-to-end medical image segmentation model. To this end, a linearly scalable alternative to self-attention is proposed by formulating a Nonnegative Matrix Factorization (NMF) algorithm as a differentiable layer. Moreover, a series of matricization operations are introduced, which enable NMF to effectively exploit both global and local contexts. The Factorizer block is constructed by replacing the self-attention layer of a ViT block with our NMF-based modules and then integrated into a U-shaped architecture with skip connections.

We evaluate the effectiveness of our approach on the task of brain tumor segmentation in MRI data. Factorizes achieve competitive results on the BraTS dataset [34, 35, 36] and outperform state-of-the-art methods based CNN and Transformer. Our experiments show that NMF components are highly meaningful, which gives a great advantage to Factorizers over CNNs and Transformers in terms of interpretability. Furthermore, our ablation studies reveal a distinctive interesting feature of Factorizers that enables us to easily speed up the inference for a trained Factorizer model with no extra steps and

without sacrificing much accuracy.

Contributions. The main contributions of this work are as follows:

- To the best of our knowledge, this work presents the first end-to-end deep model with matrix factorization layers for medical image segmentation.
- We propose scalable interpretable U-Net-like models based on NMF for medical image segmentation.
- We construct a differentiable layer performing NMF with a block coordinate descent solver to efficiently model contextual information.
- Shifted Window (Swin) Matricize operation is proposed and combined with NMF to effectively exploit local contexts.
- The proposed models achieve state-of-the-art results on the BraTS dataset.

Notation. Lets briefly overview notation. We denote vectors by boldface lower-case letters, e.g., $\mathbf{x}$, matrices by boldface upper-case letters, e.g., $\mathbf{X}$, and tensors by boldface calligraphic letters, e.g., $\boldsymbol{\mathcal{X}}$. Elements in a matrix (tensor) are denoted by $\mathbf{X}[i,j]$ ($\boldsymbol{\mathcal{X}}[i_1,\ldots,i_N]$). The ith row and jth column of a matrix is denoted by $\mathbf{X}[i,:]$ and $\mathbf{X}[:,j]$, respectively. A sequence of N vectors (a.k.a *tokens*) is denoted by $(\mathbf{x}_n)_{n=1}^N$. We use $[\mathbf{x}_1|\ldots|\mathbf{x}_N]$ to denote a matrix $\mathbf{X}$ created by stacking $\mathbf{x}_i$s along the columns. We show the inner product between matrices by $\langle \mathbf{X}, \mathbf{Y} \rangle = \sum_{i,j} \mathbf{X}[i,j]\mathbf{Y}[i,j]$ and ℓ_2-norm of a matrix by $\|\mathbf{X}\| = \sqrt{\langle \mathbf{X}, \mathbf{X} \rangle}$.

2. Related Work

CNN-based Segmentation Models. Convolution neural networks (CNNs) have dominated medical image segmentation. Particularly, following an encoder-decoder architecture with skip connections, U-Net [1] has achieved state-of-the-art on various medical image datasets. The simplicity and effectiveness of U-shaped architecture have made numerous U-Net variants emerge into the field. Çiçek et al. [2] extended U-Net by replacing all 2D operations with their 3D counterparts. UNet++ [7] follows a deeply-supervised encoder-decoder network consisting of sub-networks connected through a series of nested, dense skip connections. nnU-Net [8] proved effective in various medical image segmentation tasks by only making minor modifications to the standard 3D U-Net [2] and defining a recipe to automatically configure key design choices. Myronenko [9] proposed a U-Net-like architecture with ResNet blocks, a.k.a ResSegNet, winning first place in the Brain Tumor Segmentation Challenge (BraTS) 2018.

Despite their success, these networks generally fail to effectively model long-range spatial dependencies, often necessary for better recognition of some region semantics such as tumors, since they rely on convolution operations with small kernel sizes, aggregating only local information in an image.

Visual Transformers. Transformers with attention mechanisms [10], introduced originally for language modeling, have recently proved promising on computer vision tasks. Particularly, the pioneering Vision Transformer (ViT) model [11] achieved great performance on image recognition by large-scale pre-training and fine-tuning a pure Transformer applied to sequences of image patches, outperforming state-of-the-art CNNs. In

contrast to CNNs, ViT lacks any inductive bias, such as locality, and therefore generally shows poorer performance than CNN counterparts (e.g., ResNets [12]) when trained from scratch on small-size or mid-size datasets, which is usually the case for medical imaging.

Efforts have been made to mitigate this limitation. For example, Tokens-to-Token ViT (T2T-ViT) [13] introduces a hierarchical architecture to ViT by progressively combining neighboring tokens into a single token to reduce the sequence length and aggregate local context. Liu et al. [14] proposed a hierarchical Transformer, called Swin Transformer, adopting the shifted windowing scheme, which brings more efficiency by limiting self-attention computation to non-overlapping local windows while also allowing for cross-window connection. Pyramid vision Transformer (PvT) [15] is another exemplification of hierarchical Transformer, which relies on a non-overlapping patch partition to reduce the sequence length and a linear patch embedding to keep the channel dimensions consistent. Moreover, PvT uses spatial-reduction attention (SRA) in each block to significantly reduce the computational cost by learning low-resolution key-value pairs. Convolutional vision Transformer (CvT) [16] incorporates depthwise convolutions into self-attention layers and uses strided convolution for simultaneously tokenizing and downsampling the image, exploiting the excellent capability of convolution at capturing low-level local features.

Transformer-based methods were recently proposed to deal with the task of 2D image segmentation. Segmentation Transformer (SETR) [17] uses a ViT encoder and a decoder with progressive upsampling (which alternates Conv layers and upsampling operations) and multi-Level feature Aggregation. SegFormer [18] consists of a PvT-based encoder and a simple MLP-based decoder with upsampling operations. Chen et al. [19] proposed a model for multi-organ segmentation by incorporating ViT into the bridge of a 2D convolutional U-Net architecture. Zhang et al. [20] proposed to combine a shallow CNN with a Transformer in a parallel style. Valanarasu et al. [21] proposed a Transformer model with an axial attention mechanism for the segmentation of 2D medical images. Cao et al. [22] proposed a U-shaped architecture based purely on Swin Transformer.

For 3D medical image segmentation, Xie et al. [23] proposed a model comprising a CNN backbone to extract features, a Transformer to model long-range dependencies, and a CNN decoder to construct the segmentation map. More recently, Hatamizadeh et al. [24] proposed UNETR, which utilizes ViT as the main encoder but directly connects it to the convolutional decoder via skip connections, as opposed to using a Transformer only in the bridge. Since self-attention is prohibitively expensive on long sequences, all these models apply Transformer on a low-resolution level after either patch embedding or a CNN backbone, making them fail to fully exploit the global context at the higher resolutions. In contrast, our proposed approach based on NMF offers a scalable alternative to attention mechanism, which enables the exploitation of global context at the highest-resolution level of a 3D network.

Matrix Factorization Models. In the context of machine learning, matrix factorization (MF) methods have proved extremely useful for representation learning, dimensionality reduction, and collaborative filtering. Nonnegative Matrix Factorization (NMF) has been used for unsupervised and semi-automated segmentation of brain tumors on multiparametric MRI data [25, 26]. However, there exist only a few works that incorporate MF into an end-to-end deep model to perform a computer vision task. Most notably, Geng et al. [27] proposed a framework, called Hamburger, which models the global context as the problem of low-rank matrix completion, whose optimization algorithms can help design layers capturing global information. They demonstrated the effectiveness of a

Hamburger model based on NMF with a multiplicative update (MU) solver for semantic segmentation. Our proposed mechanism for context modeling is based on NMF with a Hierarchical Alternating Least Squares (HALS) solver, and different from Hamburger, introduces a series of matricization operations which enable NMF effectively exploit both global and local contexts. Moreover, our proposed block is inspired by the overall design of the ViT block and incorporated into a U-shaped architecture.

3. Method

3.1. Revisit Attention

The attention mechanism is the key component which enables Transformers to model complex dependencies between the elements of a sequence. Consider an input sequence of C-dimensional tokens $(\mathbf{x}_n)_{n=1}^N$, stacked into the rows of matrix $\mathbf{X} = [\mathbf{x}_1|\dots|\mathbf{x}_N]^T$. In a self-attention layer, the input is first projected onto three learnable weight matrices $\mathbf{W}^Q, \mathbf{W}^K, \boldsymbol{W}^V \in \mathbb{R}^{C\times E}$ to get three different matrices: *Query* $\mathbf{Q} = [\mathbf{q}_1|\dots|\mathbf{q}_N]^T = \mathbf{X}\mathbf{W}^Q$, *Key* $\mathbf{K} = [\mathbf{k}_1|\dots|\mathbf{k}_N]^T = \mathbf{X}\mathbf{W}^K$, and *Value* $\mathbf{V} = [\mathbf{v}_1|\dots|\mathbf{v}_N]^T = \mathbf{X}\boldsymbol{W}^V$. The output is then obtained as follows:

$$A(\boldsymbol{X}) = \text{Softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{E}}\right)\boldsymbol{V}, \tag{1}$$

where Softmax is taken row-wise. Note that attention takes $\mathcal{O}(N^2E)$ time and needs $\mathcal{O}(N^2)$ memory to store the attention map, scaling quadratically with the sequence length, which is prohibitively expensive for large inputs such a high-resolution or 3D images.

Taking a closer look at equation (1), we notice that attention can be viewed as a special case of nonparametric regression. To show this, lets consider key-value pairs $\{(\mathbf{k}_n, \mathbf{v}_n)\}_{n=1}^N$ as a training set and queries $\{\mathbf{q}_m\}_{n=1}^N$ as a test set. Predictions for the queries using a Nadaraya-Watson kernel regression [28] are:

$$\hat{f}(\mathbf{q}_n) = \frac{\sum_{m=1}^N \varphi(\mathbf{q}_n, \mathbf{k}_m)\mathbf{v}_n}{\sum_{m=1}^N \varphi(\mathbf{q}_n, \mathbf{k}_m)}, \tag{2}$$

where kernel $\varphi(.,.)$ is, in general, a similarity measure. Setting $\varphi(\mathbf{a}, \mathbf{b}) = \exp(\mathbf{a}^T\mathbf{b}/\sqrt{E})$ simple yields the attention formula.

3.2. Low-Rank Matrix Factorization

Formulating attention as a kernel regression with softmax kernel suggests that we can probably use other regression methods on queries, keys, and values to model interactions between the sequence elements. For example, one can take a matrix completion approach by low-rank decomposition of a block matrix of queries, keys, and values with missing entries, that is

$$\begin{pmatrix} \mathbf{K} & \mathbf{V} \\ \mathbf{Q} & ? \end{pmatrix} \approx \begin{pmatrix} \mathbf{B} \\ \mathbf{F} \end{pmatrix} \begin{pmatrix} \mathbf{G} \\ \mathbf{H} \end{pmatrix}^T, \tag{3}$$

where $\mathbf{B}, \mathbf{F} \in \mathbb{R}^{N\times R}$ and $\mathbf{G}, \mathbf{H} \in \mathbb{R}^{E\times R}$; and R in the rank. The reconstructed matrix $\mathbf{F}\mathbf{H}^T$ then gives an estimation of the missing block. This can be viewed as a joint matrix factorization of $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$, i.e., $\mathbf{K} \approx \mathbf{B}\mathbf{G}^T$, $\mathbf{V} \approx \mathbf{B}\mathbf{H}^T$, $\mathbf{Q} \approx \mathbf{F}\mathbf{G}^T$.

In this work, we further simplify the procedure by considering only a single linear map to generate a single matrix, e.g., query $\mathbf{Q}$, using a regular factorization rather than a joint

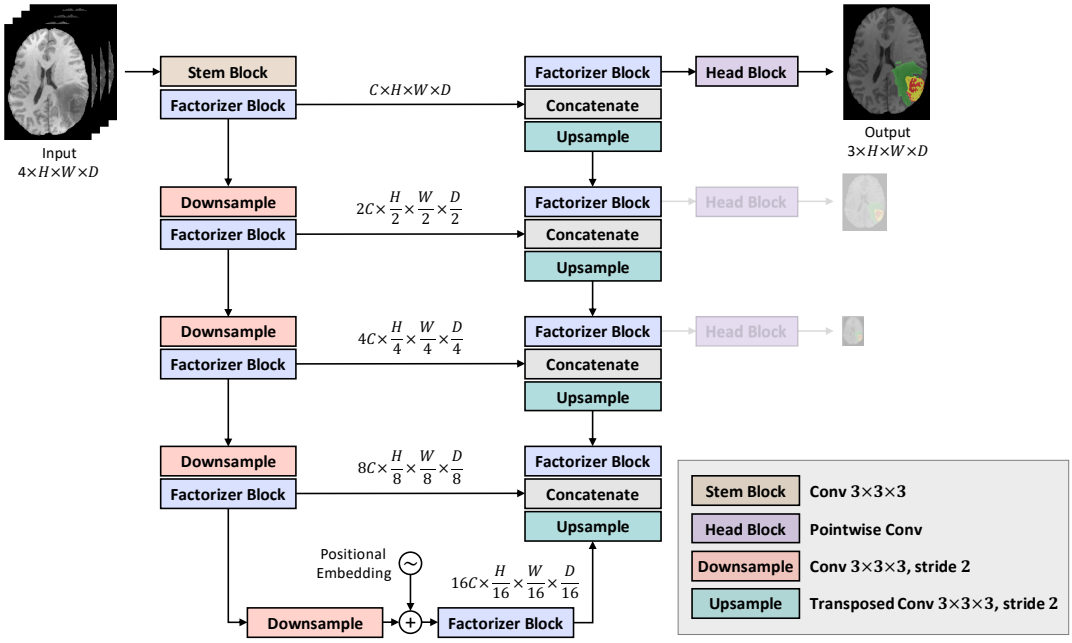


Figure 1 The overall architecture of Factorizer.

one, that is $\mathbf{Q} \approx \mathbf{F}\mathbf{G}^T$, then the output is the reconstructed matrix $\mathbf{F}\mathbf{G}^T$. Depending on the matrix factorization algorithm, a suitable activation function may be applied before the factorization to constrain the input. For example, in the case of nonnegative matrix factorization (NMF), ReLU of the matrix must be first taken to make all of the entries nonnegative.

3.3. Overall Architecture

As shown in Figure 1, a Factorizer model follows a U-Net-style architecture consisting of encoder and decoder parts. Given a 3D input image $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{C_{in} \times H \times W \times D}$ with C_{in} channels and resolution (H, W, D), the network outputs a *logit map* of size (C_{out}, H, W, D), where C_{out} is the number of segmentation classes. A $3 \times 3 \times 3$ convolution layer is used as the stem to increase the number of channels. However, note that in contrast to ViT, Factorizer does not flatten the spatial dimensions to generate a sequence of tokens.

The network has four levels, with the resolution decreasing to 1/16 in the bridge. At each level in the encoder (decoder), the input tensor is downsampled (upsampled) by a factor of two while the number of channels is doubled (halved). Convolution (transposed convolution) with kernel size $(2, 2, 2)$ and stride 2 is used for downsampling (upsampling). In the bridge, learnable position embeddings are added to the input after downsampling. We use deep supervision [29] at the three highest resolutions in the decoder, applying pointwise convolutions (i.e., convolution with kernel size $(1, 1, 1)$) to get the output and two auxiliary low-resolution logit tensors.

3.4. Factorizer Block

A Factorizer block is constructed by replacing multi-head self-attention (MSA) module in a ViT block by a Wrapped NMF module (described in Section 3.5). As shown in Figure 2a, a Factorizer block comprises NMF module and MLP, each of which follows a

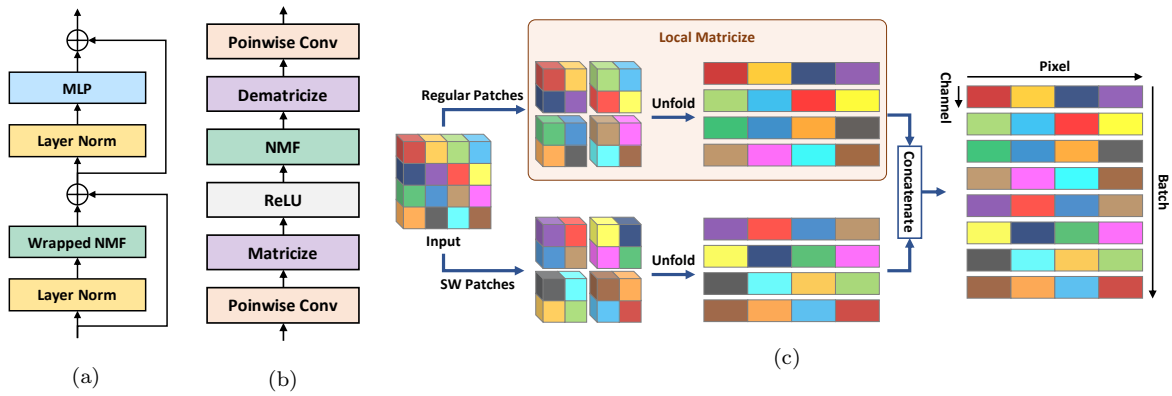


Figure 2 An overview of the Factorizer block and its components. (a) overall Factorizer block (b) Wrapped NMF module (c) An illustration of Swin Matricize on a 2D toy example.

LayerNorm and is followed by a residual connection, that is,

$$\begin{aligned}\mathcal{Z}' &= \text{WrappedNMF}(\text{LayerNorm}(\mathcal{X}) + \mathcal{X},\\ \mathcal{Z} &= \text{MLP}(\text{LayerNorm}(\mathcal{Z}')) + \mathcal{Z}',\end{aligned} \tag{4}$$

where MLP has two linear layers with GELU nonlinearity in between:

$$\text{MLP}(\mathcal{Z}) = \text{PointwiseConv}(\text{GELU}(\text{PointwiseConv}(\mathcal{X})). \tag{5}$$

The number of input and output channels are the same, but the number of inner channels is set twice that of the input channels.

3.5. Wrapped NMF Module

The integral component of Factorizer is the Wrapped NMF module, which relies on matricization (i.e., an operation that turns a tensor into a matrix) and nonnegative matrix factorization (NMF). As shown in 2b, a Wrapped NMF subblock first applies a pointwise convolution to linearly project each voxel. The output is then reshaped into a batch of matrices using a *Matricize* operation, which is detailed later on in Section 3.5.1. The resulting matrices pass through ReLU to clamp all their elements into nonnegative values and then low-rank approximated using NMF. The constructed matrices are reshaped back to their original size using the *Dematricize* operation, i.e., the inverse of Matricize. Finally, another pointwise convolution is applied, yielding the output. More formally, Wrapped NMF can be computed as follows:

$$\begin{aligned}\mathcal{X}^1 &= \text{PointwiseConv}(\mathcal{X}),\\ \mathcal{X}^2 &= \text{Matricize}(\mathcal{X}^1),\\ \mathcal{X}^3 &= \text{NMF}(\text{ReLU}(\mathcal{X}^2)),\\ \mathcal{X}^4 &= \text{Dematricize}(\mathcal{X}^3),\\ \text{WrappedNMF}(\mathcal{X}) &= \text{PointwiseConv}(\mathcal{X}^4),\end{aligned} \tag{6}$$

where intermediate tensors $\mathcal{X}^i$s and the output have the same size as that of input $\mathcal{X}$.

3.5.1. Matricize

Before applying any matrix factorization method, an input batch of multi-dimensional images, denoted by $\mathcal{X} \in \mathbb{R}^{B\times C\times H\times W\times D}$, must be turned into a batch of matrices, say

$\mathbf{Z} \in \mathbb{R}^{B' \times M \times N}$. This operation is called *Matricize*. In this work, we propose three Matricize operations: i) Global Matricize, ii) Local Matricize, and iii) Shifted Window (SW) Matricize. Depending on which operation is used, different variants for Factorizer are obtained: i) Global Factorizer, ii) Local Factorizer, and iii) **S**hifted **Win**dow (Swin) Factorizer. The Matricize operations are described in the following.

Global Matricize This operation simply flattens the spatial dimensions and also divides the channels into multiple groups (analogous to heads of Multi-Head Self-Attention). More specifically, Global Matricize reshapes a batch of C-channel 3D images $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{B \times C \times H \times W \times D}$ into a batch of matrices denoted by a 3D tensor $\mathbf{Z} \in \mathbb{R}^{B(C/E) \times E \times HWD}$, where E is called the head dimension, i.e., the number of channels per head (or matrix). This operation is obviously suitable for modeling global context and imposes no locality inductive bias.

Local Matricize Global Matricize lacks a notion of locality which is typically useful for images, especially in low-data regimes. Local Matricize is proposed to mitigate this shortcoming by splitting an input $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{B \times C \times H \times W \times D}$ into a grid of non-overlapping patches of size (E, P, P, P). These patches are flattened spatially and then concatenated along the batch dimension, yielding a batch of matrices presented by the tensor $\mathbf{Z} \in \mathbb{R}^{BCHW/(EP^3) \times E \times P^3}$. The entire procedure can be summarized by Einstein notation:

$$\mathbf{Z}[bg_c g_h g_w g_d, e, p_e p_w p_d] = \boldsymbol{\mathcal{X}}[b, g_c e, g_h p_h, g_w p_w, g_d p_d] \tag{7}$$

where (g_c, g_h, g_w, g_d) and (e, p_h, p_w, p_d) indices correspond to grid and patch dimensions, respectively. In PyTorch and TensorFlow, such Einstein operations can be simply implemented using `rearrange` function from `einops` library. Once NMF is applied to the resulting batch of matrices, the inverse operation of Local Matricize, called Local Dematricize, must be applied to transform them back to the initial shape. Local Dematricize can be easily obtained by composing the inverses of sub-operations in reverse order. In practice, Local Dematricize can also be formulated and implemented as an Einstein operation. A PyTorch-style pseudocode of Local (De)matricize module is provided in Algorithm 3.

Note that while Local Matricize seems to reshape an image in a similar way to the input patchifier of ViT, it concatenates patches along batch dimension rather than channel dimension. In fact, Local Matricize can be used when modeling within-patch interactions is desirable, which is different from ViT-based approaches, where interactions between embedded patches are typically modeled.

Shifted Window Matricize. While Local Matricize introduces locality to a model, voxels close to the boundaries of partitioning windows are not represented effectively. Since patches are low-rank approximated independently later on in an NMF layer, two neighboring boundary voxels from two adjacent patches are very likely to end up having excessively different feature maps in the output of the factorizer block, which in turn eventually makes the prediction less accurate for such voxels. To mitigate this problem, we utilize a shifted window approach proposed by Liu et al. [14] and introduce a Shifted Window (SW) Matricize operation, making the output feature maps smoother around boundaries.

An illustration of SW Matricize is provided in Figure 2c. Here, in addition to regular patches similar to those extracted by Local Matricize, shifted window (SW) patches are also included. Let's $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{B \times C \times H \times W \times D}$ be the input and (P, P, P) the patch size. To

extract SW patches, the input must be first shifted by the offset of $(P/2, P/2, P/2)$ along spatial dimensions such that voxels shifted beyond the boundaries of the images are re-introduced at the first position, yielding a tensor of the same size $B \times C \times H \times W \times D$. We call this operation *Roll* (which can be implemented using `roll` function in PyTorch and TensorFlow). The resulting rolled tensor is then reshaped by Local Matricize to get SW patches. Finally, both the batches of regular and SW patches are concatenated along the batch dimension. Formally, SW Matricize is computed as follows:

$$\begin{aligned}
\boldsymbol{\mathcal{X}}^1 &= \text{LocalMatricize}_P(\boldsymbol{\mathcal{X}}), \\
\boldsymbol{\mathcal{X}}^2 &= \text{LocalMatricize}_P(\text{Roll}_{P/2}(\boldsymbol{\mathcal{X}})), \\
\text{SWMatricize}(\boldsymbol{\mathcal{X}}) &= \text{Concatenate}(\boldsymbol{\mathcal{X}}^1, \boldsymbol{\mathcal{X}}^2),
\end{aligned} \tag{8}$$

where LocalMatricize_P denotes Local Matricize with a patch size of (P, P, P), and $\text{Roll}_{P/2}$ denotes Roll operator with a shift $(P/2, P/2, P/2)$. To build SW Dematricize, we need to reconstruct the image from both regular and SW patches independently then compute their average to achieve a smoother and more accurate feature maps. Further details are provided in Appendix B, which includes a PyTorch implementation of SW (De)matricize module presented in Algorithm 3.

3.5.2. Nonnegative Matrix Factorization

Once the input is somehow transformed to a batch of matrices, and its negative elements are clipped to zero by ReLU, it is ready to be low-rank approximated by Nonnegative Matrix Factorization (NMF). This is the main component of a factorizer model that contributes most to modeling local or global context in an image.

NMF [30] seeks to approximate some given nonnegative matrix $\mathbf{X} \in \mathbb{R}_{\geq 0}^{M \times N}$ by

$$\mathbf{X} \approx \mathbf{F}\mathbf{G}^T, \tag{9}$$

where $\mathbf{F} \in \mathbb{R}_{\geq 0}^{M \times R}$ and $\mathbf{G} \in \mathbb{R}_{\geq 0}^{N \times R}$ are factors, and the positive integer $R \leq \min(M, N)$ is the rank. Once the factors $\mathbf{F}$ and $\mathbf{G}^T$ are somehow approximated, the NMF layer in a Factorizer block outputs the reconstructed matrix $\hat{\mathbf{X}} = \mathbf{F}\mathbf{G}^T$. Various loss functions have been used to form the objective function and measure the quality of an approximation. Depending on the loss function, constraints, and regularization, many variants of NMF have been proposed. In this work, we use a standard NMF with the squared error, which is the most-widely used variant for images, to find factors by solving

$$\underset{\mathbf{F} \geq 0, \mathbf{G} \geq 0}{\text{minimize}} \ \|\mathbf{X} - \mathbf{F}\mathbf{G}^T\|^2. \tag{10}$$

In general, problem (10) is nonconvex, and finding global minima is NP-hard. However, numerous iterative algorithms have been proposed to find a "good local minimum". The majority of existing methods are based on a block coordinate descent (BCD) scheme (a.k.a alternating optimization), where the objective function is iteratively minimized with respect to one factor while the other factor is kept fixed. That is, the convex subproblems

$$\mathbf{F} \leftarrow \underset{\mathbf{F} \geq 0}{\arg\min} \ \|\mathbf{X} - \mathbf{F}\mathbf{G}^T\|^2, \qquad \mathbf{G} \leftarrow \underset{\mathbf{G} \geq 0}{\arg\min} \ \|\mathbf{X}^T - \mathbf{G}\mathbf{F}^T\|^2, \tag{11}$$

are exactly or approximately solved alternately.This ensures that the objective function value does not increase after each update and guarantees convergence to a stationary point under some mild conditions [31].

Algorithm 1: Multiplicative update solver for NMF ($\circ$ denotes element-wise matrix product).

Input: $\mathbf{X} \in \mathbb{R}_{\geq 0}^{M \times N}$ (input matrix), R (rank); T (# iterations)
Output: $\hat{\mathbf{X}} \in \mathbb{R}_{\geq 0}^{M \times N}$ (low-rank approximated matrix)

Initialize factors $\mathbf{F} \in \mathbb{R}_{\geq 0}^{M \times R}$ and $\mathbf{G} \in \mathbb{R}_{\geq 0}^{N \times R}$
for $t = 1, \ldots, T$ **do**
  $\mathbf{F} \leftarrow \mathbf{F} \circ \dfrac{\mathbf{X}\mathbf{G}}{\mathbf{F}\mathbf{G}^T\mathbf{G}}$
  $\mathbf{G} \leftarrow \mathbf{G} \circ \dfrac{\mathbf{X}^T\mathbf{F}}{\mathbf{G}\mathbf{F}^T\mathbf{F}}$
return $\hat{\mathbf{X}} = \mathbf{F}\mathbf{G}^T$

Algorithm 2: Hierarchical alternating least squares for NMF.

Input: $\mathbf{X} \in \mathbb{R}_{\geq 0}^{M \times N}$ (input matrix), R (rank); T (# iterations)
Output: $\hat{\mathbf{X}} \in \mathbb{R}_{\geq 0}^{M \times N}$ (low-rank approximated matrix)

Initialize factors $\mathbf{F} \in \mathbb{R}_{\geq 0}^{M \times R}$ and $\mathbf{G} \in \mathbb{R}_{\geq 0}^{N \times R}$
for $t = 1, \ldots, T$ **do**
  $\mathbf{A} = \mathbf{X}\mathbf{G}$, $\mathbf{B} = \mathbf{G}^T\mathbf{G}$
  for $r = 1, \ldots, R$ **do**
    $\mathbf{F}[:, r] \leftarrow \max(0, \dfrac{\mathbf{A}[:, r] - \sum_{\ell \neq r} \mathbf{B}[\ell, r]\mathbf{F}[:, \ell]}{\|\mathbf{G}[:, r]\|^2})$
  $\mathbf{A} = \mathbf{X}^T\mathbf{F}$, $\mathbf{B} = \mathbf{F}^T\mathbf{F}$
  for $r = 1, \ldots, R$ **do**
    $\mathbf{g}[:, r] \leftarrow \max(0, \dfrac{\mathbf{A}[:, r] - \sum_{\ell \neq r} \mathbf{B}[\ell, r]\mathbf{G}[:, \ell]}{\|\mathbf{F}[:, r]\|^2})$
return $\hat{\mathbf{X}} = \mathbf{F}\mathbf{G}^T$

Among numerous BCD-based algorithms for NMF, Multiplicative Update (MU) [30] is the best-known due to the advantage of being easy-to-implement and scalable. MU enforces nonnegativity constraint by updating the previous values of a factor matrix by multiplication with a nonnegative scale factor. A pseudocode of MU is pretended in Algorithm 1. However, slow convergence of MU has been pointed [32], and hence more effective algorithms, such as Hierarchical Alternating Least Squares (HALS) [33], with faster convergence have been introduced. HALS updates a factor matrix, say $\mathbf{F} = [\mathbf{f}_1 | \ldots | \mathbf{f}_R]$, with inner iterations, in which the columns $\mathbf{f}_r$s are updated by solving the following subproblem

$$\mathbf{f}_r \leftarrow \underset{\mathbf{f}_r \geq 0}{\arg\min} \|\mathbf{E}_r - \mathbf{f}_r\mathbf{g}_r^T\|^2, \tag{12}$$

where $\mathbf{E}_r = \mathbf{X} - \sum_{\ell \neq r}^{R} \mathbf{f}_\ell \mathbf{g}_\ell^T$ is the residual matrix, which is, in fact, approximated by a rank-one matrix. An encouraging aspect of HALS is that each subproblem (12) can be easily shown to have a closed-form solution:

$$\mathbf{f}_r^* \leftarrow \max(0, \frac{\mathbf{E}_r\mathbf{g}_r}{\|\mathbf{g}_r\|^2}). \tag{13}$$

Similarly, the update formula for columns of $\mathbf{G}$ can be derived. Note that HALS is a $2R$-block coordinate descent procedure, where at each outermost iteration, first the columns of $\mathbf{F}$ and then the columns of $\mathbf{G}$ are updated. Algorithm 2 provides pseudocode of HALS (further details on MU and HALS can be found in [32], Chapter 8).

A special case of NMF is $R = 1$; i.e., $\mathbf{X} \approx \mathbf{f}\mathbf{g}^T$, where $\mathbf{f} \in \mathbb{R}^M_{\geq 0}$ and $\mathbf{g} \in \mathbb{R}^N_{\geq 0}$; for which one can easily investigate that both MU and HALS are simplified to the same update rule:

$$\mathbf{f} \leftarrow \frac{\mathbf{X}\mathbf{g}}{\|\mathbf{g}\|^2}, \qquad \mathbf{g} \leftarrow \frac{\mathbf{X}\mathbf{f}}{\|\mathbf{f}\|^2}. \tag{14}$$

In this paper, all Factorizer models are trained with $R = 1$, and compression ratios (and somewhat reconstruction errors) are controlled by adjusting the head dimension (i.e., the number of rows in a matrix, as discussed in 3.5.1) to sufficiently small values. This means that a matricize operation transforms an image into a batch of such fat matrices (whose number of columns are far more than rows) that rank-one approximation would suffice in practice. This greatly simplifies a factorizer model and improves the interpretability while yielding better performance in segmentation. However, in one of our ablation studies (see Section 4.5), we experimented with both MU and HALS for $R > 1$ to investigate the impact of rank in the inference phase.

It is worth mentioning that not all NMF algorithms and their settings can be used in the NMF layer of a Factorizer block. The utilized algorithm must have some properties so that we can ultimately train the Factorizer model successfully in an end-to-end fashion using a gradient descent-based optimizer on GPU(s) in an existing deep learning framework, such as PyTorch. Firstly, the algorithm should be backpropagation-friendly and amenable to automatic differentiation, that is $\frac{\partial \hat{\mathbf{X}}}{\partial \mathbf{X}}$ should not only be well-defined and somewhat smooth but also computable by an existing deep learning framework, such as PyTorch, so that we can practically train the Factorizer model in an end-to-end fashion using a gradient descent optimizer. Another related aspect is that the gradient $\frac{\partial \hat{\mathbf{X}}}{\partial \mathbf{X}}$, as explained in [27], stats to vanish during backpropagation after some iterations. Therefore, the number of outer iterations T in an NMF algorithm should be limited in order to have stable gradients. For MU and HALS, $T = 5$ is a reasonable choice in practice. Finally, update rules should be also friendly to GPU parallel processing for exploiting GPUs to train Factorizers in a reasonable amount of time. Taking all these factors into account, MU and HALS are appropriate choices. While HALS has better convergence properties, MU is more favorable for GPU training due to unparallelizable inner iterations of HALS.

4. Experiments

4.1. dataset.

We evaluate the effectiveness of our models in segmenting brain tumors on the Brain Tumour Segmentation (BraTS) dataset [34, 35, 36] from Medical Segmentation Decathlon (MSD) (Task 1) [37]. The BraTS training set consists of 484 multiparametric MRI (mpMRI) scans from patients diagnosed with either low-grade glioma or high-grade glioma (glioblastoma). Each scan comes with four 3D MRI sequences, namely T2 Fluid-Attenuated Inversion Recovery (FLAIR), native T1-weighted (T1), post-Gadolinium contrast T1-weighted (T1gd), and T2-weighted (T2). Once images are prepossessed; i.e., rigidly co-registered the same anatomical template, resampled to the same voxel spacing 1mm^3, and skull-stripped; the ground truths are manually created by experts who

label each voxel as enhancing tumor (ET), edema (ED), necrotic and non-enhancing tumor (NCR/NET), or everything else. However, for evaluation, the 3 nested subregions, namely enhancing tumor (ET), tumor core (TC–i.e., the union of ED and NCR/NET), and whole tumor (WT) are used (see the sample ground truths in Figure 5)

4.2. Setup

All the models were implemented using PyTorch [38] and MONAI [39] frameworks and trained on NVIDIA P100 GPUs. We followed the same training workflow in all the experiments. In the following, we first provide the details of this workflow and baseline models, then present the evaluation protocol and the results.

Preprocessing. For each scan in the BraTS dataset, a 4-channel 3D image as the input was first constructed by concatenating the four modalities, FLAIR, T1, T1gd, and T2. The image and its ground truth were then cropped with a minimal box filtering out zero regions. The image was normalized channel-wise using z-score to have intensities with zero mean and unit variance. Random patches of size $(128, 128, 128)$ were extracted from the images for training. To reduce overfitting, we used data augmentation techniques, including random affine transform, random flip along each spatial dimension, additive Gaussian noise, random Gaussian smoothing, random scale intensity, random shift intensity, and random gamma transform. The further details are provided in Appendix A.

Training. All models were trained for 100k steps with batch size 2 (one sample per GPU) using AdamW optimizer with a base learning rate of 1e−4, weight decay of 1e−2, warmup of 2k steps, and cosine annealing scheduler. The loss function is a combination of *soft Dice loss* [40] and cross-entropy loss, defined as

$$\mathcal{L}(\mathbf{G}, \mathbf{Y}) = \mathcal{L}_{\text{Dice}}(\mathbf{G}, \mathbf{Y}) + \mathcal{L}_{\text{CE}}(\mathbf{G}, \mathbf{Y}), \tag{15}$$

where

$$\mathcal{L}_{\text{Dice}}(\mathbf{G}, \mathbf{P}) = 1 - \frac{2\langle \mathbf{G}, \mathbf{Y} \rangle}{\|\mathbf{G}\|^2 + \|\mathbf{Y}\|^2}, \qquad \mathcal{L}_{\text{CE}}(\mathbf{G}, \mathbf{Y}) = -\frac{1}{N}\langle \mathbf{G}, \log(\mathbf{Y}) \rangle, \tag{16}$$

where $\mathbf{G} \in \{0, 1\}^{J \times N}$ and $\mathbf{Y} \in [0, 1]^{J \times N}$ represent the one-hot encoded ground truth and the predicted probability map for each voxel, respectively, with J denoting the number of segmentation classes and N denoting the number of voxels in the patch. The three deep supervision outputs and the corresponding downsampled ground truths are used for computing the final loss. The training objective is the weighted mean of the losses at all these resolutions, with the weights being 1, 0.5, and 0.25 at resolutions 128^3, 64^3, and 32^3, respectively.

Inference. A test image in the inference was first subjected to z-score intensity normalization, then the prediction was made using a sliding window approach with a 50% overlap and a window size of $128 \times 128 \times 128$ (same as the patch size used in training). Finally, the resulting probabilities were thresholded by 0.5 to obtain a binary segmentation map.

Evaluation Metrics. The Dice score and Hausdorff Distance 95% (HD95) were used as metrics to asses the performance of models in our experiments. For each segmentation

region, Dice score measures the voxel-wise overlap between the ground truth and the prediction, defined as

$$\text{Dice}(\mathbf{g}, \mathbf{p}) = \frac{2\sum_{n=1}^{N} g_n p_n}{\sum_{n=1}^{N} g_n + \sum_{n=1}^{N}, p_n} \tag{17}$$

where $g_n \in \{0, 1\}$ and $p_n \in \{0, 1\}$ represent the ground truth and the binary prediction for a voxel, respectively, and N is the number of voxels. If both the ground truth and the prediction do not have any nonzero values, that is the denominator of equation (17) is zero, the Dice score is defined as 1. Hausdorff Distance (HD) evaluates the distance between the boundaries of ground truth and prediction. HD is defined as follows:

$$\text{HD}(G, P) = \max\{\max_{\mathbf{g}\in G} \min_{\mathbf{p}\in P} \|\mathbf{g} - \mathbf{p}\|, \max_{\mathbf{p}\in P} \min_{\mathbf{g}\in G} \|\mathbf{p} - \mathbf{g}\|\}, \tag{18}$$

where G and P denote the set of all voxels on the surface of ground truth and prediction, respectively. HD95 is a more robust version to outliers, which calculates the 95% quantile rather than the maximum of surface distances.

Models. In all the experiments, the number of output channels of the stem block was 32. The patch size of Local and SW Matricize was set to $(8, 8, 8, 8)$. In NMF modules, the factor matrices were initialized with uniform distribution $\mathcal{U}(0, 1)$. In training, we used a rank-one approximation ($R = 1$) with $T = 5$ outer iterations of HALS (which is equivalent to MU for $R = 1$), as described in Section 3.5.2.

To make a fair comparison between factorizer and baseline models, we neutralize the effect of architectural variability by keeping the overall architecture of baselines the same as that of Factorizer and only changing encoder and decoder blocks. This helps to better investigate the impact of blocks rather than architectures. Three models are used as baselines, all of which follow the same overall architecture illustrated in Figure 1, except that each has a different encoder and decoder block. The baselines are detailed in the following:

- U-Net: This model is based on 3D U-Net [2] with minor modifications. Each encoder (and decoder) block is composed of two convolutions with a kernel size of $(3, 3, 3)$. Each convolutional layer is followed by LeakyReLU nonlinearity and Group Normalization (with a group size of 8) in order. This model does not have any positional embedding in its bridge since CNNs already have some notion of position. This model is very similar to nnU-Net [41]

- Res-U-Net: ResNet block [12] is the cornerstone of Res-U-Net. This block is similar to that of U-Net, except that it has residual connection after the last Group Normalization. This model is very similar to SegResNet [9].

- Performer: The encoder and decoder blocks of this model are based on ViT (note that the input is first flattened into a sequence of voxels before feeding to a ViT block). However, attention scales quadratically with the number of voxels, hence is prohibitively expensive in our case, where the input can have up to 128^3 voxels. Therefore, we replace original attention layers of a ViT block with FAVOR+ (Fast Attention Via positive Orthogonal Random features) used in Performer, recently proposed by Choromanski et al. [42] as a linearly scalable alternative to Transformer. FAVOR+ gives an unbiased estimate of attention using only linear (as opposed to quadratic) time and space complexity. Note that except for the attention layers, the rest of the components of this baseline are the same as those of Factorizer.

Table 1 Comparison of different models on the BraTS dataset. Scores are obtained by 5-Fold cross-validation. The asterisks indicate how significantly our models scores differ from those of baselines according to pairwise Wilcoxon signed-rank test (*: p-value < 0.05; **: p-value < 0.01; ***: p-value < 0.001; and ****: p-value < 0.0001).

Model	#Params	FLOPs	Dice (%)				Hausdorff 95% (mm)			
			ET	TC	WT	Avg.	ET	TC	WT	Avg.
U-Net	28.7M	1264.7G	75.89	81.94	90.09	82.64	8.61	9.13	9.31	9.19
Res-U-Net	28.9M	1280.8G	77.95	82.49	**90.39***	83.61	6.08	8.18	**8.06****	7.52
Performer	6.7M	321.8G	78.43	82.34	88.72	83.16	6.72	9.78	12.93	10.21
Global Factorizer	5.9M	282.2G	78.20	82.86	88.65	83.24	6.67	9.53	11.69	9.71
Local Factorizer	5.9M	283.9G	78.67	82.85	89.30	83.61	5.29	7.77	8.91	7.41
Swin Factorizer	5.9M	290.4G	**79.33**	**83.14*****	90.16	**84.21****	**4.91****	**7.31***	8.23	**6.89***

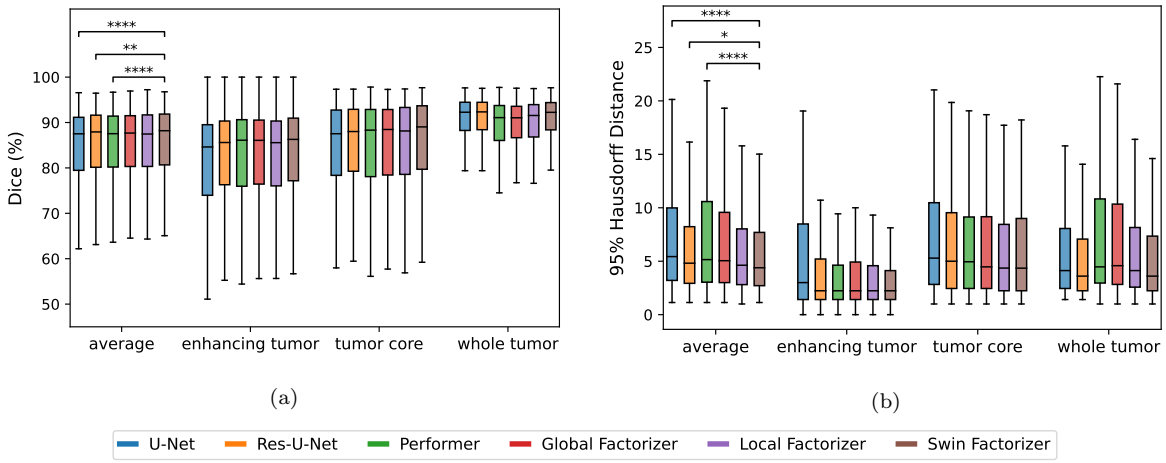
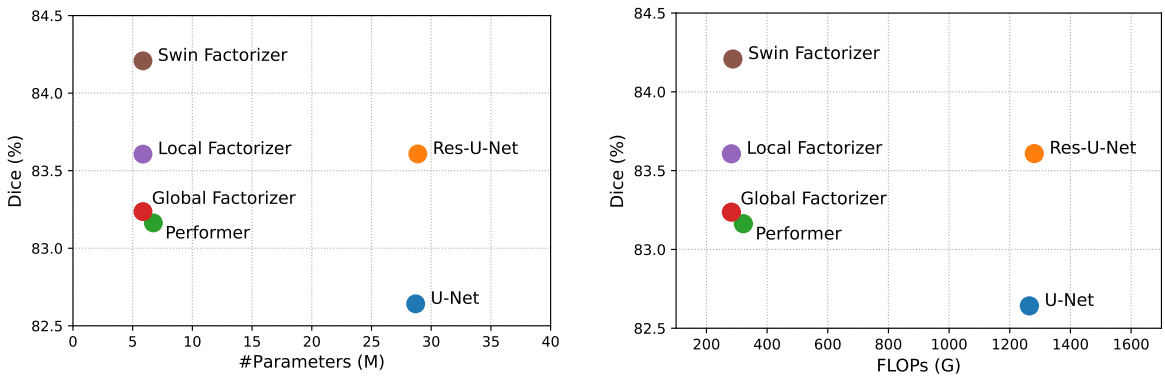


Figure 3 The Dice score and 95% Hausdorff distance of different models on the BraTS datasest. The asterisks indicate statistical significance according to pairwise Wilcoxon signed-rank test (*: p-value < 0.05; **: p-value < 0.01; ***: p-value < 0.001; and ****: p-value < 0.0001).

4.3. Results

Quantitative Evaluation. For all the experiments, we performed 5-fold cross-validation to estimate how capable our models are in generalizing to unseen data. Pairwise Wilcoxon signed-rank test was also for comparing the performance of our models with that of the baselines. The results on the BraTS dataset are reported in Table 1 and illustrated by box plots in Figure 3.

Swin Factorizer was superior to all the other models, which achieved an average Dice of 84.21% and HD95 of 6.89, significantly outperforming Res-U-Net, the best baseline, with p-values of < 0.01 for average Dice and < 0.05 for average HD95, while requiring substantially fewer computations. Swin Factorizer was the best-performing on ET and TC subregions, particularly yielding a Dice of 83.14% on TC, which is significantly higher than the others with p-value < 0.001. Local Factorizer was our second-best model, and its performance was comparable with that of Res-U-Net. More specifically, Local Factorizer outperformed Res-U-Net on ET and TC while underperforming on WT. As shown in Figure 4, while achieving competitive performance over U-Net and Res-U-Net, Factorizer models have much fewer parameters and are significantly cheaper, taking over four times lower theoretical computational cost (FLOPs).



(a) (b)

Figure 4 Performance of Factorizer models in terms of average Dice on BraTS, versus their speed in terms of the number of FLOPs (left) and versus the number of parameters (right).

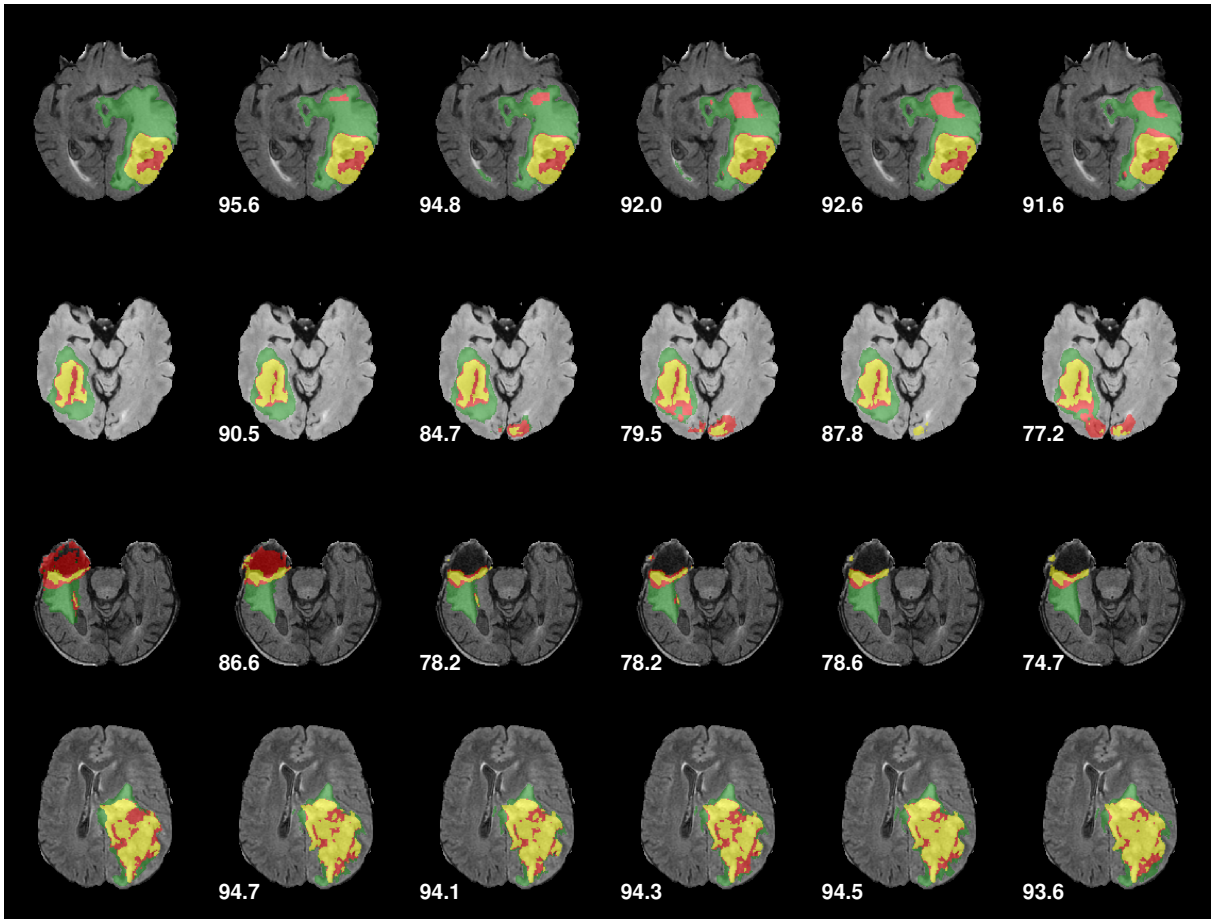
Performer is the Transformer-based counterpart of Global Factorizer in the sense that they both only exploit global context without imposing any locality inductive bias in their blocks. However, while having lower computational complexity, Global Factorizer, with an average Dice score of 83.24% and an average HD95 of 9.71, marginally outperformed Performer, with an average Dice score of 83.16% and an average HD95 of 10.21. Local Factorizer improved the performance of Global Factorizer by exploiting locality, particularly with the average HD95 significantly dropping from 9.71 to 7.41 (p-value < 0.0001). As expected, Swin Factorizer improved all the scores of Local Factorizer, which is consistent with the fact that Swin Factorizer modifies Local Factorizer by better representing the boundary voxels in Local Matricize.

Qualitative Comparisons. Qualitative comparisons of glioma segmentation models are presented in Figure 5. Swin Factorizer demonstrates superior performance in segmenting TC and ET. This capability is evident in row 1 (row 2), where Swin Factorizer more successfully delineates TC compared to the other models, which do not as accurately distinguish edema (normal tissues) from tumor core. Particularly, the CNN-based models, i.e., U-Net and Res-U-Net, misclassify a significantly larger part of edema (normal tissues) as necrosis (enhancing tumor). Row 3 exemplifies a successful detection of TC by Swin Factorizer, while the other models miss a fairly large necrotic region.

In comparison to Performer (the Transformer-based model), Global Factorizer exhibits slightly higher accuracy in identifying tumor cores, especially differentiating necrosis from edema or normal regions, as seen in rows 1 and 2. As expected, compared to Global Factorizer, Swin Factorizer demonstrates a higher level of ability to capture the fine-grained details of enhancing tumor and necrotic regions.

4.4. NMF Components Interpretability

One additional advantage of Factorizer over Transformer and CNN models is its higher interpretability resulting from meaningful components of NMF. Note that both the first and last Factorizer blocks have $C = 32$ channels, divided into groups of 8-channel heads during matricization, where the NMF of each head has only a single rank-one term, i.e., $R = 1$. As a result, in total, there are $CR/8 = 4$ components in both the first and last blocks. Figure 6b and 6c show the components of the first and last NMF layers on a high-grade glioma case for Swin Factorizer and Global Factorizer. More precisely,



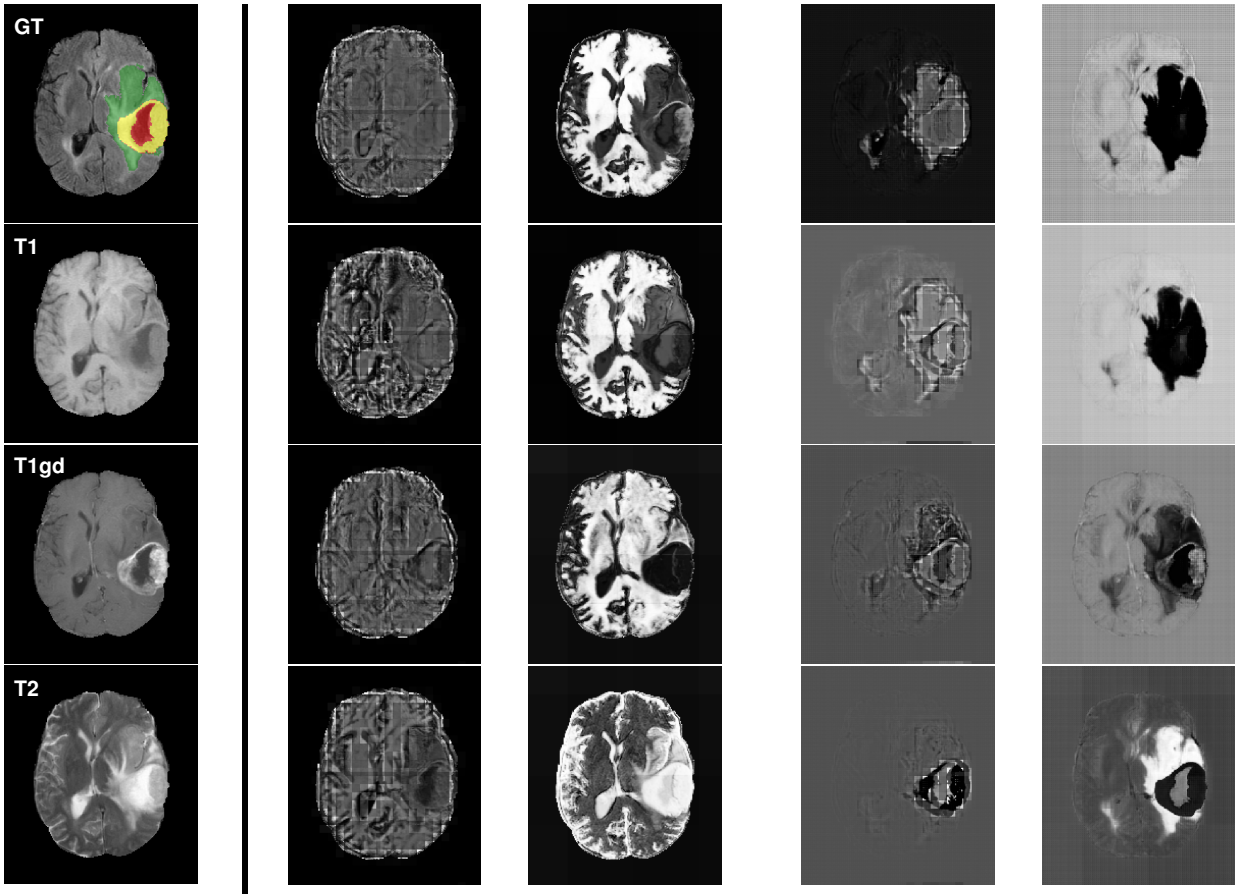
Ground Truth **Swin Factorizer** **Global Factorizer** **Performer** **Res-U-Net** **U-Net**

Figure 5 Qualitative results on brain tumor segmentation. ET is denoted in yellow, TC is the union of red and yellow regions, and WT includes the union of green, red, and yellow regions. The average Dice score is presented for each subject.

each component illustrates the factor matrix corresponding to spatial dimensions after dematricization.

Interestingly, the components are highly meaningful and interpretable in the sense that each component gives an interpretation by differentiating one region from another. For instance, as observed in Figure 6b, the first (row 1) and second (row 2) components discriminate roughly the whole tumor while the third (row 3) and fourth (row 4) components capture the tumor core. For Swin Factorizer, necrosis is distinguished more clearly in the third and fourth components whereas for Global Factorizer necrosis is more recognizable in the second component. As we get to deeper layers, components become even more meaningful such that in the last layer, each component clearly detects some regions. For example, as seen in Figure 6c, the first (row 1) and second (row 2) components very clearly discriminate the whole tumor while the third (row 3) and fourth (row 4) components also capture the tumor core and necrosis.

Another interesting observation is that Global Factorizer yields more discriminative and higher-level components in the first layer, which can be attributed to the fact Global Factorizer models long-range dependencies through all of its blocks, including the first one; however, the receptive field of Swin Factorizer in the first stage of the encoder is small but progressively increases as it passes through downsampling layers. Therefore,



(a) Modalities (b) Components of first NMF layer (c) Components of last NMF layer

Figure 6 NMF Components of Factorizer models on BraTS. Panel (a) shows the modalities and ground truth for a high-grade glioma case. Panel (b) and (c) present the NMF spatial components in the first and last block, receptively.

Swin Factorizer, similar to CNNs, extracts lower-level features in shallower layers and higher-level ones in deeper layers. Notice that the footprints of sliding windows, which appear as a grid pattern, are also evident in all the components of Swin Factorizer,

4.5. Ablation Studies

4.6. Ablation Studies in Training

We conducted ablation studies on Factorizer to further investigate the effects of Factorizer subblocks and positional embedding. In this section, models with an ablated layer were trained from scratch on the BraTS dataset.

Factorizer Subblock. Ablation results of subblocks on brain tumor segmentation are reported in Table 2. When we removed NMF subblocks from a Factorizer model, the performance substantially dropped while Factorizer models without MLP subblocks demonstrated lesser deterioration in results. Local Factorizer without MLP blocks yielded an average Dice of 82.67%, still outperforming U-Net, and Swin Factorizer without MLP blocks yielded an average Dice of 83.57%, which is significantly greater than that of U-Net and comparable with that of Res-U-Net. These results indicate the effectiveness of

Table 2 The impact of each sublock on the performance of Factorizers on BraTS.

MLP Subblock	NMF Subblock	Matricize	Dice (%)				Hausdorff 95% (mm)			
			ET	TC	WT	Avg.	ET	TC	WT	Avg.
✓	✗	✗	77.02	80.89	87.36	81.76	6.55	8.90	11.29	9.02
✗	✓	Global	77.51	80.78	87.85	82.05	8.82	11.24	16.18	12.33
✗	✓	Local	77.79	81.67	88.56	82.67	5.50	8.67	9.44	8.04
✗	✓	SW	78.60	82.61	89.51	83.57	5.00	7.47	7.86	6.87
✓	✓	SW	79.33	83.14	90.16	84.21	4.91	7.31	8.23	6.89

Table 3 The impact of positional embedding on the performance of Factorizers on BraTS (the asterisks indicate p-value < 0.05 in the pairwise Wilcoxon signed-rank test between a model with and without positional embedding).

Model	Positional Embedding	Dice (%)				Hausdorff 95% (mm)			
		ET	TC	WT	Avg.	ET	TC	WT	Avg.
Global Factorizer	✗	77.98	82.54	88.76	83.09	7.57	9.91	11.73	10.12
Global Factorizer	✓	78.20	82.86	88.65	**83.24***	6.67	9.53	11.69	**9.71**
Local Factorizer	✗	78.24	82.59	89.37	83.40	5.87	8.75	9.98	8.31
Local Factorizer	✓	78.67	82.85	89.30	**83.61**	5.29	7.77	8.91	**7.41**
Swin Factorizer	✗	78.63	83.18	90.16	83.99	5.50	7.24	8.03	6.97
Swin Factorizer	✓	79.33	83.14	90.16	**84.21**	4.91	7.31	8.23	**6.89**

NMF layer in improving the models.

Positional Embedding. Table 3 shows the results of the ablation study on positional embedding. In all the cases, adding positional embedding to the bridge of a network improved the performance. Particularly, the average Dice score of Global Factorizer increased from 83.09% to 83.24% significantly with p-value $= 0.031$, and the average HD95 fell from 10.12 to 9.71 although this improvement is not statistically significant. Local and Swin Factorizers without positional embedding yielded an average Dice of 83.40% and 83.99%, respectively, slightly underperforming compared to their counterparts with positional embedding. Similar to Transformers, Factorizers lack any notion of voxels position, and therefore typically benefit from a positional embedding mechanism, which is generally consistent with our results.

4.7. Ablation Studies in Inference

Since the output of an NMF layer is a low-rank approximation of the input, it makes sense to perform an ablation study in the inference phase, for instance by short-circuiting the NMF layer. For all the experiments of this section, we ablated some NMF layers or change their settings in the inference phase after training the model on the BraTS dataset.

NMF Layer. We investigated the impact of short-circuiting some NMF layers of the pre-trained Swin Factorizer model in the inference phase. In Figure 7a, the average Dice score on BraTS is shown when we kept the first NMF layers and removed (or short-circuit) the rest. As expected, the more layers we kept, the higher the Dice score was

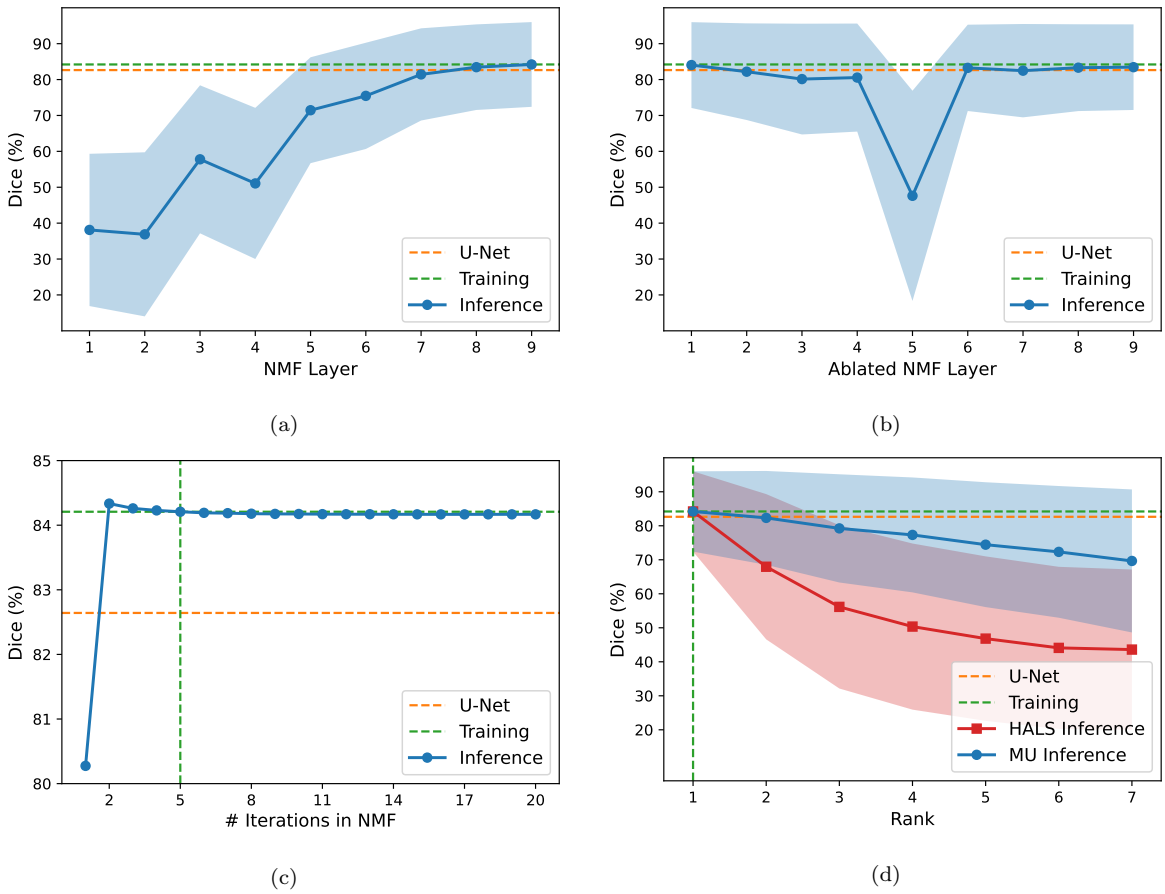


Figure 7 Inference-phase ablation experiments with Swin Factorizer pre-trained on BraTS. Panel (a) shows the average Dice score when the first NMF layers are only kept, while Panel (b) shows the Dice score plotted versus the NMF layer removed. Panel (c) shows the plot of Dice against the number of outer iterations in HALS. Panel (d) shows the impact of rank on the performance in HALS and MU. The dashed orange lines indicate the U-Net Dice score, while the dashed green lines correspond to the pre-trained Swin Factorizer model.

obtained. We observed that most of the performance is achieved via the encoder, which includes the first five blocks. Figure 7b shows the results of investigating the impact of each individual layer, where we kept all the layers except one at a time. Interestingly, we noticed that the NMF layer of the bride block (layer 5) makes the greatest contribution to the performance. In fact, if an NMF layer except that of the bridge is ablated, the average Dice still stays above 80%. Particularly, if an NMF layer in the decoder (layers 6 to 9) is ablated, the performance is still better than that of U-Net. Note that removing an NMF layer, especially those at higher-resolution stages of the network, can significantly reduce the computational complexity and speed up the inference time.

NMF Solver Iterations. We investigate the effect of changing the number of outer iterations (T) in HALS. Recall that all the Factorizer models were originally trained using HALS with $T = 5$. Figure 7c shows the results of experimenting with $T \in \{1, \ldots, 20\}$ in the inference phase for the Swin Factorizer model pre-trained with $T = 5$. We noticed that for $T > 1$, the performance is very close to that of the original model ($T = 5$). Interestingly, $T \in \{2, 3, 4\}$ yielded even higher average Dice scores compared to the original model although the improvement was not statistically significant. We attribute this to the possible regularization effect of reducing T. Note that we proportionally

decreased the computational cost of NMF layers by reducing T while preserving the accuracy. This brings a great advantage to Factorizers over CNNs and Transformers, when it comes to model speed-up (discussed further in Section 5).

Rank. We investigated the impact of changing the rank of the pre-trained Swin Factorizer model in the inference phase. Recall that Swin Factorizers were trained with $R = 1$, in which both HALS and MU lead to the same update rule. Therefore, for $R > 1$ in inference, we experimented with both HALS and MU in order to make a comparison. In Figure 7d, the average Dice score on BraTS is plotted as a function of rank. As observed, the more the rank deviated from $R = 1$ (the one used in training), the more significantly the Dice score dropped. In comparison to HALS, MU demonstrated a less dramatic reduction in the performance as we increased the rank. This can be due to the fact that within the same number of outer iterations, HALS typically makes a larger decrease in the NMF objective (given in equation (10)), causing the HALS-based model to deviate further from the original model than that of MU.

5. discussion

Here, we elaborate on some key features distinguishing Factorizer from other approaches. Similar to self-attention, the Wrapped NMF layer can be viewed as an adaptive filter, meaning that the computation of factor matrices involve the input, as opposed to convolution, where kernel weights are fixed and independent from the input and do not change after training. For Local and Swin Factorizer, we used a large window size of $(8, 8, 8)$ to aggregate local information, in contrast to typical CNNs comprising $3 \times 3 \times 3$ convolutions. The Wrapped NMF layer scales linearly and is much cheaper than self-attention with quadratic complexity (which is computationally intractable on long sequences) and even Performer, as an efficient approximation of attention. Moreover, by training a Factorizer, we also obtain lightweight yet accurate versions of that for free without needing to re-train any model from scratch. All we need is to reduce T or ablate some costly NMF layers in the inference phase. As shown in Section 4.5, this did not sacrifice much accuracy on the BraTS dataset and even slightly improved the performance when reducing T to values greater than 2, probability due to its regularization impact. However, CNNs and Transformers lack such a desirable feature, requiring much more complex mechanisms to achieve their faster versions.

6. Future Work

Matrix factorization models are very flexible and versatile. In this work, we used an ordinary NMF model together with some matricization techniques to model local or global context. One possible future work is to customize the NMF objective to simultaneously exploit both local and global contexts. Moreover, it would be useful to explore some ideas for automated selection of the rank hyperparameter, for example, by taking a greedy approach to NMF, a.k.a Nonnegative Matrix Underapproximation, where the components are constructed and added one by one (sequentially) until some criteria are met. This can benefit especially Local Factorizer as different regions happen to have different optimal ranks depending on their contexts. Finally, we will explore the effectiveness of other NMF variants, such as Semi-NMF and Convex-NMF, which work on mixed-signed data matrices and relax the need for the ReLU activation function before factorization.

7. Conclusion

Vision Transformers, particularly those with hierarchical architectures, have recently achieved results comparable with state-of-the-art CNNs on various computer vision tasks. Nevertheless, lack of locality inductive bias makes them underperform their CNN counterparts in low-data regimes, which is usually the case in medical image segmentation. Moreover, the quadratic complexity of attention makes exiting Transformers apply self-attention layers only after somehow reducing the image resolution, thus fail to fully capture long-range contexts present at higher resolutions. Hence, this paper introduces a series of models, dubbed Factorizer, which leverage the power of low-rank approximation for developing a scalable interpretable approach to context modeling by formulating Nonnegative Matrix Factorization (NMF) as a differentiable layer integrated into an end-to-end U-shaped architecture. Build upon NMF and shifted window idea, Swin Factorizer competed favorably with CNN and Transformer baselines in terms of accuracy and scalability, achieving state-of-the-art results on the BraTS dataset for brain tumor segmentation. Our experiments indicated that NMF components are highly meaningful, which gives a great advantage to Factorizers over CNNs and Transformers in terms of interpretability. Moreover, our ablation studies revealed a distinctive feature of Factorizers that allows the speed-up of inference for a pre-trained Factorizer with no extra steps and without sacrificing much accuracy.

Acknowledgements. The research leading to these results has received funding from EU H2020 MSCA-ITN-2018: INtegrating Magnetic Resonance SPectroscopy and Multimodal Imaging for Research and Education in MEDicine (INSPiRE-MED), funded by the European Commission under Grant Agreement #813120. This research also received funding from the Flemish Government (AI Research Program). Sabine Van Huffel and Pooya Ashtari are affiliated to Leuven.AI - KU Leuven institute for AI, B-3000, Leuven, Belgium.

Appendix

A. Data Augmentation Pipeline

The MONAI library [39] was used for data augmentation. The following random transforms were applied on the fly during training in order:

1. **Affine transform**. An affine transform was applied with a probability of 0.15, where the angles of rotation (in degrees) and the scaling factor were drawn from $\mathcal{U}(-30, 30)$ and $\mathcal{U}(0.7, 1.3)$, respectively.

2. **Flip**. All patches were flipped with a probability of 0.5 along each spatial dimension.

3. **Gaussian Noise**. Each voxel of a patch was independently subject to additive white Gaussian noise with a variance drawn from $\mathcal{U}(0, 0.1)$. This augmentation was applied with a probability of 0.15.

4. **Gaussian Smoothing**. Each modality of a patch was filtered independently via a symmetric Gaussian kernel whose width (in voxels) was drawn from $\mathcal{U}(0.5, 1.5)$. This augmentation was applied with a probability of 0.15.

5. **Scale Intensity**. Voxel intensities were scaled with $s \sim \mathcal{U}(0.7, 1.3)$ with a probability of 0.15.

6. **Shift Intensity**. Voxel intensities were shifted with offsets drawn from $\mathcal{U}(-0.1, 0.1)$ with a probability of 0.15.

7. **Adjust Contrast**. Gamma correction with $\gamma \sim \mathcal{U}(0.7, 1.5)$ was applied voxel-wise with a probability of 0.15.

B. Matricize Implementation Details

Algorithm 3 presents PyTorch implementations of Local and Swin Matricize modules. Here, we use the `einops` library to reshape tensors simply by declaring the corresponding Einstein notations.

Algorithm 3: PyTorch Pseudocode of Local and Swin Matricize modules.

```python
import torch
from torch import nn
from einops.layers.torch import Rearrange


class LocalMatricize(nn.Module):
    def __init__(self, size, head_dim=8, patch_size=(8, 8, 8)):
        super().__init__()
        C, H, W, D = size
        c2, (h2, w2, d2) = head_dim, patch_size
        c1, h1, w1, d1 = C // c2, H // h2, W // w2, D // d2
        dims = dict(c1=c1, h1=h1, w1=w1, d1=d1, c2=c2, h2=h2, w2=w2, d2=d2)

        eq = "b (c1 c2) (h1 h2) (w1 w2) (d1 d2) -> (b c1 h1 w1 d1) c2 (h2 w2 d2)"
        self.unfold = Rearrange(eq, **dims)

        eq = "(b c1 h1 w1 d1) c2 (h2 w2 d2) -> b (c1 c2) (h1 h2) (w1 w2) (d1 d2)"
        self.fold = Rearrange(eq, **dims)

    def forward(self, x):
        # x: (B, C, H, W, D)
        return self.unfold(x)

    def inverse_forward(self, x):
        # x: (B, C, L)
        return self.fold(x)


class SwinMatricize(nn.Module):
    def __init__(
        self, size, head_dim=8, patch_size=(8, 8, 8), shifts=(4, 4, 4)
    ):
        super().__init__()
        self.local_matricize = LocalMatricize(size, head_dim, patch_size)
        self.shifts = shifts
        self.shifts_inv = tuple(-s for s in patch_size)

    def forward(self, x):
        # x: (B, C, H, W, D)
        out1 = self.local_matricize(x)
        out2 = torch.roll(x, self.shifts, (2, 3, 4))
        out2 = self.local_matricize(out2)
        return torch.cat((out1, out2))

    def inverse_forward(self, x):
        # x: (B, C, L)
        B = x.shape[0]
        out1 = self.local_matricize.inverse_forward(x[: (B // 2)])
        out2 = self.local_matricize.inverse_forward(x[(B // 2) :])
        out2 = torch.roll(out2, self.shifts_inv, (2, 3, 4))
        return 0.5 * (out1 + out2)
```

References

[1] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *International Conference on Medical image computing and computer-assisted intervention*. Springer, 2015, pp. 234–241.

[2] Ö. Çiçek, A. Abdulkadir, S. S. Lienkamp, T. Brox, and O. Ronneberger, "3d u-net: learning dense volumetric segmentation from sparse annotation," in *International conference on medical image computing and computer-assisted intervention*. Springer, 2016, pp. 424–432.

[3] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs," *IEEE transactions on pattern analysis and machine intelligence*, vol. 40, no. 4, pp. 834–848, 2017.

[4] Z. Gu, J. Cheng, H. Fu, K. Zhou, H. Hao, Y. Zhao, T. Zhang, S. Gao, and J. Liu, "Ce-net: Context encoder network for 2d medical image segmentation," *IEEE transactions on medical imaging*, vol. 38, no. 10, pp. 2281–2292, 2019.

[5] X. Wang, R. Girshick, A. Gupta, and K. He, "Non-local neural networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 7794–7803.

[6] J. Fu, J. Liu, H. Tian, Y. Li, Y. Bao, Z. Fang, and H. Lu, "Dual attention network for scene segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 3146–3154.

[7] Z. Zhou, M. M. R. Siddiquee, N. Tajbakhsh, and J. Liang, "Unet++: A nested u-net architecture for medical image segmentation," in *Deep learning in medical image analysis and multimodal learning for clinical decision support*. Springer, 2018, pp. 3–11.

[8] F. Isensee, P. F. Jaeger, S. A. Kohl, J. Petersen, and K. H. Maier-Hein, "nnu-net: a self-configuring method for deep learning-based biomedical image segmentation," *Nature methods*, vol. 18, no. 2, pp. 203–211, 2021.

[9] A. Myronenko, "3d mri brain tumor segmentation using autoencoder regularization," in *International MICCAI Brainlesion Workshop*. Springer, 2018, pp. 311–320.

[10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in neural information processing systems*, 2017, pp. 5998–6008.

[11] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.

[12] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[13] L. Yuan, Y. Chen, T. Wang, W. Yu, Y. Shi, Z. Jiang, F. E. Tay, J. Feng, and S. Yan, "Tokens-to-token vit: Training vision transformers from scratch on imagenet," *arXiv preprint arXiv:2101.11986*, 2021.

[14] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," *arXiv preprint arXiv:2103.14030*, 2021.

[15] W. Wang, E. Xie, X. Li, D.-P. Fan, K. Song, D. Liang, T. Lu, P. Luo, and L. Shao, "Pyramid vision transformer: A versatile backbone for dense prediction without convolutions," *arXiv preprint arXiv:2102.12122*, 2021.

[16] H. Wu, B. Xiao, N. Codella, M. Liu, X. Dai, L. Yuan, and L. Zhang, "Cvt: Introducing convolutions to vision transformers," *arXiv preprint arXiv:2103.15808*, 2021.

[17] S. Zheng, J. Lu, H. Zhao, X. Zhu, Z. Luo, Y. Wang, Y. Fu, J. Feng, T. Xiang, P. H. Torr *et al.*, "Rethinking semantic segmentation from a sequence-to-sequence perspective with transformers," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 6881–6890.

[18] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "Segformer: Simple and efficient design for semantic segmentation with transformers," *arXiv preprint arXiv:2105.15203*, 2021.

[19] J. Chen, Y. Lu, Q. Yu, X. Luo, E. Adeli, Y. Wang, L. Lu, A. L. Yuille, and Y. Zhou, "Transunet: Transformers make strong encoders for medical image segmentation," *arXiv preprint arXiv:2102.04306*, 2021.

[20] Y. Zhang, H. Liu, and Q. Hu, "Transfuse: Fusing transformers and cnns for medical image segmentation," *arXiv preprint arXiv:2102.08005*, 2021.

[21] J. M. J. Valanarasu, P. Oza, I. Hacihaliloglu, and V. M. Patel, "Medical transformer: Gated axial-attention for medical image segmentation," *arXiv preprint arXiv:2102.10662*, 2021.

[22] H. Cao, Y. Wang, J. Chen, D. Jiang, X. Zhang, Q. Tian, and M. Wang, "Swin-unet: Unet-like pure transformer for medical image segmentation," *arXiv preprint arXiv:2105.05537*, 2021.

[23] Y. Xie, J. Zhang, C. Shen, and Y. Xia, "Cotr: Efficiently bridging cnn and transformer for 3d medical image segmentation," *arXiv preprint arXiv:2103.03024*, 2021.

[24] A. Hatamizadeh, Y. Tang, V. Nath, D. Yang, A. Myronenko, B. Landman, H. R. Roth, and D. Xu, "Unetr: Transformers for 3d medical image segmentation," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2022, pp. 574–584.

[25] N. Sauwen, M. Acou, S. Van Cauter, D. Sima, J. Veraart, F. Maes, U. Himmelreich, E. Achten, and S. Van Huffel, "Comparison of unsupervised classification methods for brain tumor segmentation using multi-parametric mri," *NeuroImage: Clinical*, vol. 12, pp. 753–764, 2016.

[26] N. Sauwen, M. Acou, D. M. Sima, J. Veraart, F. Maes, U. Himmelreich, E. Achten, and S. Van Huffel, "Semi-automated brain tumor segmentation on multi-parametric mri using regularized non-negative matrix factorization," *BMC medical imaging*, vol. 17, no. 1, pp. 1–14, 2017.

[27] Z. Geng, M.-H. Guo, H. Chen, X. Li, K. Wei, and Z. Lin, "Is attention better than matrix decomposition?" *arXiv preprint arXiv:2109.04553*, 2021.

[28] W. K. Härdle, M. Müller, S. Sperlich, and A. Werwatz, *Nonparametric and semi-parametric models*. Springer Science & Business Media, 2004.

[29] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang, and Z. Tu, "Deeply-supervised nets," in *Artificial intelligence and statistics*. PMLR, 2015, pp. 562–570.

[30] D. D. Lee and H. S. Seung, "Learning the parts of objects by non-negative matrix factorization," *Nature*, vol. 401, no. 6755, pp. 788–791, 1999.

[31] L. Grippo and M. Sciandrone, "On the convergence of the block nonlinear gauss–seidel method under convex constraints," *Operations research letters*, vol. 26, no. 3, pp. 127–136, 2000.

[32] N. Gillis, *Nonnegative Matrix Factorization*. SIAM, 2020.

[33] A. Cichocki and A.-H. Phan, "Fast local algorithms for large scale nonnegative matrix and tensor factorizations," *IEICE transactions on fundamentals of electronics, communications and computer sciences*, vol. 92, no. 3, pp. 708–721, 2009.

[34] B. H. Menze, A. Jakab, S. Bauer, J. Kalpathy-Cramer, K. Farahani, J. Kirby, Y. Burren, N. Porz, J. Slotboom, R. Wiest *et al.*, "The multimodal brain tumor image segmentation benchmark (brats)," *IEEE transactions on medical imaging*, vol. 34, no. 10, pp. 1993–2024, 2014.

[35] S. Bakas, H. Akbari, A. Sotiras, M. Bilello, M. Rozycki, J. S. Kirby, J. B. Freymann, K. Farahani, and C. Davatzikos, "Advancing the cancer genome atlas glioma mri collections with expert segmentation labels and radiomic features," *Scientific data*, vol. 4, p. 170117, 2017.

[36] S. Bakas, M. Reyes, A. Jakab, S. Bauer, M. Rempfler, A. Crimi, R. T. Shinohara, C. Berger, S. M. Ha, M. Rozycki *et al.*, "Identifying the best machine learning algorithms for brain tumor segmentation, progression assessment, and overall survival prediction in the brats challenge," *arXiv preprint arXiv:1811.02629*, 2018.

[37] M. Antonelli, A. Reinke, S. Bakas, K. Farahani, B. A. Landman, G. Litjens, B. Menze, O. Ronneberger, R. M. Summers, B. van Ginneken *et al.*, "The medical segmentation decathlon," *arXiv preprint arXiv:2106.05735*, 2021.

[38] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, pp. 8026–8037, 2019.

[39] N. Ma, W. Li, R. Brown, Y. Wang, Behrooz, B. Gorman, H. Johnson, I. Yang, E. Kerfoot, charliebudd, Y. Li, M. Adil, Y.-T. Hsieh, A. Aggarwal, C. Trentz, adam aji, masadcv, M. Graham, B. Murray, G. Daroach, P.-D. Tudosiu, myron, M. McCormick, Ambros, Balamurali, C. Baker, J. Sellner, L. Fidon, and cgrain, "Project-monai/monai: 0.5.3," Jun. 2021. [Online]. Available: https://doi.org/10.5281/zenodo.4891800

[40] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-net: Fully convolutional neural networks for volumetric medical image segmentation," in *2016 fourth international conference on 3D vision (3DV)*. IEEE, 2016, pp. 565–571.

[41] F. Isensee, P. Kickingereder, W. Wick, M. Bendszus, and K. H. Maier-Hein, "No new-net," in *International MICCAI Brainlesion Workshop*. Springer, 2018, pp. 234–244.

[42] K. Choromanski, V. Likhosherstov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser *et al.*, "Rethinking attention with performers," *arXiv preprint arXiv:2009.14794*, 2020.